Java Backend Architecture

Interview Series

Chapter 13.10 Database Performance Tuning

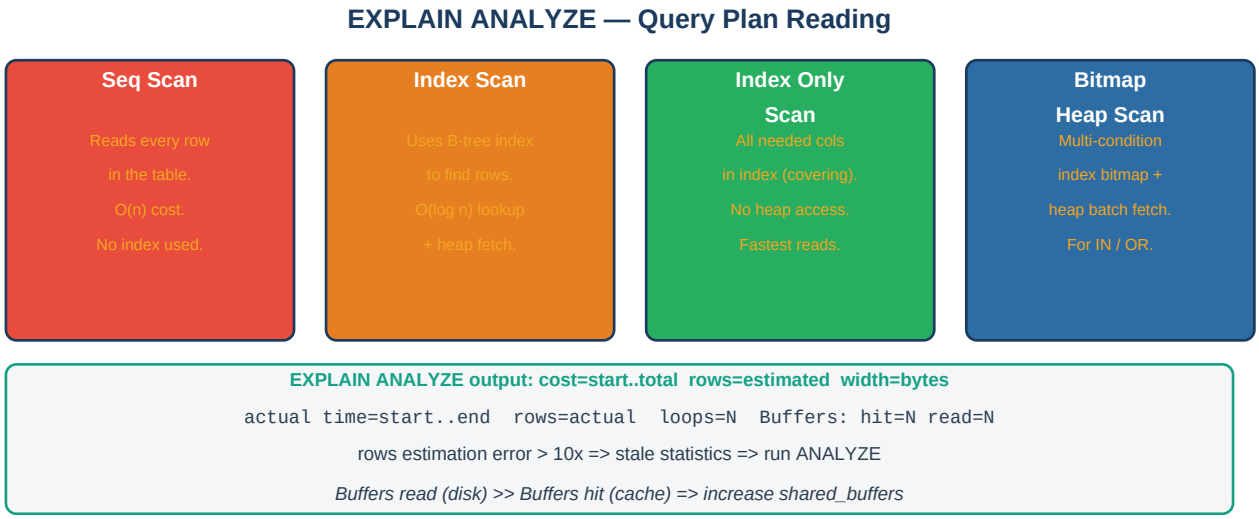
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Database Performance Tuning

Database performance is frequently the primary bottleneck in Java backend systems. Understanding how the database plans and executes queries — via EXPLAIN ANALYZE — is the foundation of performance tuning. Index selection (B-tree, Hash, GIN for JSONB/arrays, BRIN for time-series) determines whether the database scans the entire table or navigates directly to matching rows. Advanced index features — partial indexes (WHERE clause filters) and covering indexes (INCLUDE columns) — enable Index Only Scans that never touch the heap, delivering the fastest reads. Index bloat from heavy UPDATE/DELETE workloads reduces index efficiency and requires periodic REINDEX to reclaim performance.

Application-level patterns also critically affect database performance. The N+1 query problem — fetching N entities then issuing a separate query per entity for related data — causes catastrophic load; it is solved with JOIN FETCH or @EntityGraph in JPA. Connection pooling (HikariCP in application, PgBouncer between application and database) multiplexes many application threads onto a smaller number of database connections, reducing connection overhead. pg_stat_statements identifies the slowest queries in production by cumulative execution time. Autovacuum maintains table statistics and reclaims dead tuples. Read replicas scale read-heavy workloads by routing SELECT queries to replica nodes.

EXPLAIN ANALYZE — Query Plan Reading

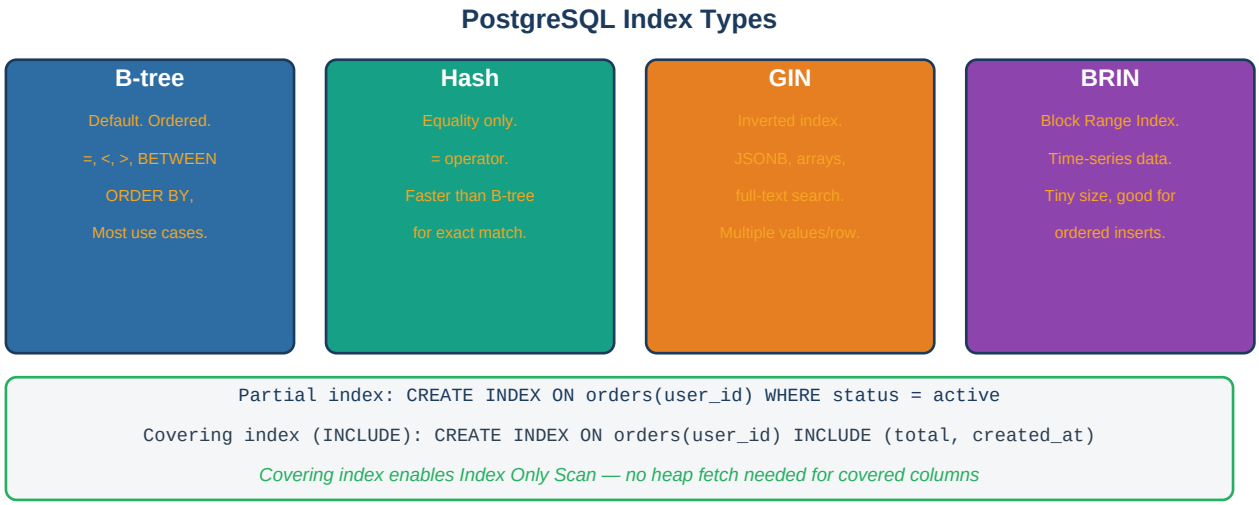


Key Definitions

EXPLAIN ANALYZE	PostgreSQL command that executes a query and returns the actual query plan with real execution times, row counts, and buffer statistics.
Seq Scan	Sequential table scan — reads every row. O(n). Used when no suitable index exists or the planner estimates index scan is slower (many matching rows).
Index Scan	Uses a B-tree (or other) index to locate rows then fetches them from the heap. Efficient for selective queries (few matching rows).
Index Only Scan	All query columns are in the index (covering index). Heap is not accessed at all. Fastest read path.

B-tree Index	Default PostgreSQL index. Stores keys in sorted order. Supports =, <, >, <=, >=, BETWEEN, ORDER BY, LIKE with prefix.
GIN Index	Generalized Inverted Index. Supports JSONB operators (@>, ?, ?|), array containment (@>), and full-text search (@@). One index entry per token/key.
BRIN Index	Block Range Index. Stores min/max value per block range. Tiny size (1000x smaller than B-tree). Effective for naturally ordered data (timestamps, sequential IDs).
Partial Index	Index with a WHERE clause, indexing only rows matching the condition. Smaller, faster, avoids indexing inactive/historical data.
Covering Index	Index using INCLUDE to add non-key columns. Enables Index Only Scan for queries that select those columns. Avoids heap fetch.
Index Bloat	Dead index entries from UPDATE/DELETE operations that have not been reclaimed. Increases index size and scan cost. Fixed by REINDEX CONCURRENTLY.
PgBouncer	Lightweight connection pooler for PostgreSQL. Transaction mode multiplexes many client connections onto fewer server connections.
N+1 Query Problem	Anti-pattern: 1 query to load N entities + N queries for related data = N+1 total queries. Solved with JOIN FETCH or @EntityGraph.
pg_stat_statements	PostgreSQL extension recording query execution statistics (total time, calls, mean time, rows). Used to identify slow queries in production.
Autovacuum	PostgreSQL background daemon that reclaims dead tuples (from UPDATE/DELETE), updates table statistics for the query planner, and prevents XID wraparound.
Read Replica	A streaming replication replica that accepts read-only queries. Reduces primary load; application routes SELECT to replica and writes to primary.

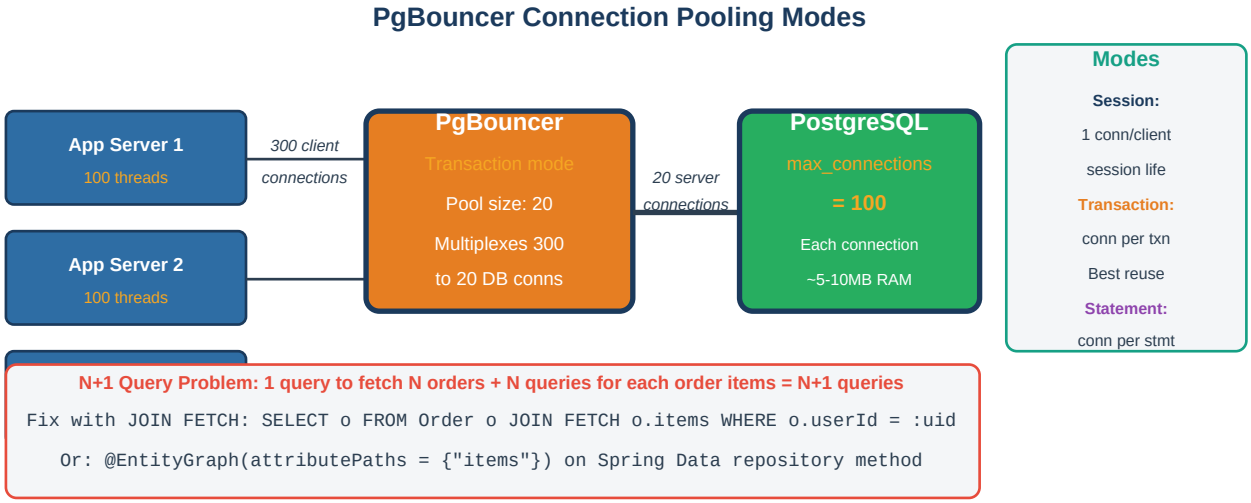
PostgreSQL Index Types



Index Type Quick Reference

Index Type	Operators Supported	Use Case	Size	Write Overhead
B-tree	=, <, >, BETWEEN, LIKE prefix, ORDER BY	Most queries	Medium	Medium
Hash	= (equality only)	Exact match lookups	Medium	Medium
GIN	@>, ?, ?|, @@, @@ (full-text)	JSONB, arrays, text search	Large	High
GiST	&&, @>, <->, within (geometry)	Spatial, range, full-text	Medium	Medium
BRIN	min/max range per block	Time-series, sequential	Very small	Very low

Connection Pooling and N+1 Query Problem



Annotated Code Examples

1. EXPLAIN ANALYZE — reading query plans

```
-- Enable buffer statistics for cache hit analysis EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT o.id, o.total, u.email FROM orders o JOIN users u ON u.id = o.user_id WHERE o.status =
'pending' AND o.created_at > NOW() - INTERVAL '7 days' ORDER BY o.created_at DESC LIMIT 100; --
Sample output to read: -- Limit (cost=120.45..120.70 rows=100 width=52) -- (actual
time=8.234..8.267 rows=100 loops=1) -- -> Sort (cost=120.45..121.45 rows=400 width=52) --
(actual time=8.231..8.244 rows=100 loops=1) -- -> Hash Join (cost=... actual time=...) -- ->
Index Scan using idx_orders_status_created -- on orders o -- (cost=0.43..85.20 rows=400
width=42) -- (actual time=0.032..4.123 rows=412 loops=1) -- Index Cond: (status = 'pending') --
Filter: (created_at > ...) -- Buffers: hit=23 read=45 <- 45 disk reads: add to cache! --
rows=400 estimated vs rows=412 actual: good estimate (< 10x off) -- If rows=400 estimated vs
rows=40000 actual: RUN ANALYZE orders;
```

2. Index design — partial, covering, and GIN

```
-- Standard B-tree index CREATE INDEX idx_orders_user_id ON orders(user_id); -- Partial index —
only index active orders (WHERE clause) -- Much smaller and faster than full-table index CREATE
INDEX idx_orders_active_user ON orders(user_id, created_at DESC) WHERE status IN ('pending',
'processing'); -- Covering index (INCLUDE) — enables Index Only Scan -- Query: SELECT id,
total, status FROM orders WHERE user_id = ? CREATE INDEX idx_orders_user_covering ON
orders(user_id) INCLUDE (id, total, status); -- included cols not part of key -- GIN index for
JSONB data CREATE INDEX idx_products_attributes_gin ON products USING gin(attributes); --
supports @> and ? operators -- Usage: SELECT * FROM products WHERE attributes @>
```

```
'{"color":"red"}'; -- BRIN index for time-series (very small, suitable for append-only data) CREATE INDEX idx_events_created_brin ON events USING brin(created_at) WITH (pages_per_range = 128); -- Rebuild bloated index without locking REINDEX INDEX CONCURRENTLY idx_orders_user_id;
```

3. N+1 query problem — JPA fix with JOIN FETCH and @EntityGraph

```
// N+1 PROBLEM: Lazy loading triggers N queries for N orders @Entity public class Order { @OneToMany(fetch = FetchType.LAZY) // loaded lazily by default private List<OrderItem> items; } // BAD: 1 query for orders + N queries for items List<Order> orders = orderRepo.findByUserId(userId); orders.forEach(o -> o.getItems().size()); // N+1 queries fired here! // FIX 1: JPQL JOIN FETCH @Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.userId = :uid") List<Order> findByUserIdWithItems(@Param("uid") Long userId); // FIX 2: @EntityGraph (declarative, no custom JPQL) @EntityGraph(attributePaths = {"items", "items.product"}) List<Order> findByUserId(Long userId); // FIX 3: Spring Data projection to fetch only needed columns interface OrderSummary { Long getId(); BigDecimal getTotal(); String getStatus(); } List<OrderSummary> findByUserId(Long userId); // no items loaded at all // Detect N+1 in development: spring.jpa.show-sql=true // Or: p6spy, datasource-proxy, Hibernate Statistics
```

4. pg_stat_statements — slow query identification

```
-- Enable extension (requires superuser, add to shared_preload_libraries) CREATE EXTENSION IF NOT EXISTS pg_stat_statements; -- Add to postgresql.conf: -- shared_preload_libraries = 'pg_stat_statements' -- pg_stat_statements.track = all -- Find top 10 slowest queries by total execution time SELECT round(total_exec_time::numeric, 2) AS total_ms, calls, round(mean_exec_time::numeric, 2) AS mean_ms, round(stddev_exec_time::numeric, 2) AS stddev_ms, rows, regexp_replace(query, '\s+', ' ', 'g') AS query FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10; -- Find queries with worst rows-per-execution (missing indexes?) SELECT query, calls, rows/calls AS avg_rows, mean_exec_time FROM pg_stat_statements WHERE calls > 100 ORDER BY rows/calls DESC LIMIT 10; -- Reset statistics after tuning SELECT pg_stat_statements_reset();
```

5. HikariCP connection pool tuning

```
# application.yml — HikariCP tuning spring: datasource: hikari: maximum-pool-size: 20 # DB connections per app instance minimum-idle: 5 # keep 5 connections idle idle-timeout: 600000 # 10 min idle before closing max-lifetime: 1800000 # 30 min max connection lifetime connection-timeout: 30000 # 30s wait for conn before error keepalive-time: 60000 # heartbeat to prevent firewall drops connection-test-query: "SELECT 1" pool-name: OrdersPool # Formula: pool_size = (cpu_cores * 2) + effective_spindle_count # For 4-core host with SSD: ~8-10 connections per application instance # With 3 app instances: 3 * 10 = 30 total connections to DB // Monitor HikariCP via Micrometer // GET /actuator/metrics/hikaricp.connections.pending // GET /actuator/metrics/hikaricp.connections.active // GET /actuator/metrics/hikaricp.connections.acquire // HikariCP recommends NOT setting minimum-idle = maximum-pool-size // for connection pools shared with PgBouncer (avoid pool-within-pool)
```

6. Autovacuum tuning and read replica routing

```
-- Check autovacuum status and dead tuple counts SELECT schemaname, tablename, n_dead_tup, n_live_tup, round(n_dead_tup * 100.0 / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct, last_autovacuum, last_autoanalyze FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10; -- Per-table autovacuum tuning (high-write tables) ALTER TABLE orders SET ( autovacuum_vacuum_scale_factor = 0.01, -- vacuum at 1% dead tuples autovacuum_analyze_scale_factor = 0.005, -- analyze at 0.5% autovacuum_vacuum_cost_delay = 2 -- reduce vacuum throttling ); // Read replica routing with Spring AbstractRoutingDataSource public class ReplicaRoutingDataSource extends AbstractRoutingDataSource { @Override protected Object determineCurrentLookupKey() { return TransactionSynchronizationManager.isCurrentTransactionReadOnly() ? "REPLICA" : "PRIMARY"; } } // Annotate read-only services @Service @Transactional(readOnly = true) // routes to REPLICA public class OrderQueryService { ... }
```

Interview Q&A;

Q1. How do you read EXPLAIN ANALYZE output?

Read from innermost indented node (executed first) outward. Each node shows: cost=start_cost..total_cost (planner estimates), rows=estimated_rows, actual time=start_ms..end_ms, rows=actual_rows, loops=N. Key signals: actual rows >> estimated rows (stale statistics, run ANALYZE); Buffers: read=N (disk I/O) >> hit=N (cache) means increase shared_buffers; sequential scans on large tables indicate missing indexes; Sort nodes with large actual time indicate missing index for ORDER BY.

Q2. What is the difference between a Seq Scan and an Index Scan?

A Seq Scan reads every row in the table (O(n)). A database uses it when: no suitable index exists, the query returns a large percentage of rows (index scan overhead exceeds sequential read benefit), or table statistics indicate low selectivity. An Index Scan uses a B-tree index to find matching rows by key, then fetches each row from the heap (random I/O). It is efficient for highly selective queries (few matching rows). An Index Only Scan skips the heap entirely when all needed columns are in the covering index.

Q3. What is a covering index and when should you use it?

A covering index includes non-key columns via INCLUDE that are needed by queries but not used for index traversal. The key columns are used for B-tree traversal (WHERE clause); the included columns are stored in leaf nodes. When a query only accesses key + included columns, PostgreSQL performs an Index Only Scan — no heap access. Use it for frequently executed SELECT queries where you know the exact column set. Trade-off: larger index size and higher write overhead.

Q4. What is a partial index and when is it beneficial?

A partial index has a WHERE clause that limits which rows are indexed. Only rows matching the condition are included. Benefits: smaller index (faster scans, better fit in cache), lower write overhead (only indexed when condition matches), more selective statistics. Use cases: indexing only active records (status = 'active'), unprocessed messages (processed = false), non-null values. A partial index on orders(user_id) WHERE status IN ('pending','processing') is much smaller than a full index.

Q5. What is index bloat and how do you fix it?

Index bloat occurs when UPDATE and DELETE operations leave dead index entries that are not immediately reclaimed. Over time the index grows larger than necessary, increasing scan time and memory usage. PostgreSQL's autovacuum reclaims dead heap tuples but index vacuum is a separate process. Fix: REINDEX INDEX CONCURRENTLY idx_name (rebuilds the index without blocking reads/writes on PostgreSQL 12+). Detect bloat with pgstattuple or by comparing index size to estimated ideal size.

Q6. What is the N+1 query problem and how do you fix it in JPA?

N+1 occurs when you load N entities (1 query) and then for each entity lazily load a related collection (N queries). For 100 orders with lazy items, you get 101 queries. Fix: (1) JPQL JOIN FETCH: "SELECT o FROM Order o JOIN FETCH o.items WHERE o.userId = :uid" — fetches orders and items in one SQL JOIN. (2) @EntityGraph on the repository method — declarative, no JPQL needed. (3) Batch fetching: @BatchSize(size=50) loads related collections in batches. (4) DTO projections: skip loading the entity graph entirely.

Q7. How does PgBouncer connection pooling work and what modes does it support?

PgBouncer sits between application and PostgreSQL, multiplexing many client connections onto fewer server connections. Session mode: each client connection holds a server connection for the entire session — minimal reuse. Transaction mode: a server connection is assigned only for the duration of a transaction,

returned to pool after COMMIT/ROLLBACK — maximum reuse, recommended for most apps. Statement mode: server connection returned after each SQL statement — cannot use transactions or server-side state. Transaction mode allows 300 application threads to share 20 PostgreSQL connections.

Q8. How do you find slow queries in production PostgreSQL?

Enable pg_stat_statements extension (shared_preload_libraries = pg_stat_statements). Query: SELECT query, total_exec_time, calls, mean_exec_time FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10 to find queries consuming the most cumulative time. Also enable log_min_duration_statement (e.g., 1000ms) to log individual slow queries. Use auto_explain extension to automatically log query plans for slow queries. In application layer, use datasource-proxy or p6spy to log slow queries from HikariCP.

Q9. What is autovacuum and why is it important?

Autovacuum is a PostgreSQL background daemon that: (1) reclaims dead tuples from UPDATE/DELETE operations (returning storage to the freespace map); (2) updates pg_statistic table statistics used by the query planner to estimate row counts; (3) prevents transaction ID wraparound (XID wraparound can cause data loss). Under-tuned autovacuum on high-write tables allows dead tuple accumulation (table bloat), stale statistics (bad query plans), and growing table size. Tune with per-table autovacuum_vacuum_scale_factor and autovacuum_vacuum_cost_delay.

Q10. How do you use read replicas to scale read-heavy workloads?

Configure PostgreSQL streaming replication with one or more replicas. In Spring Boot, use AbstractRoutingDataSource with two DataSource beans (PRIMARY and REPLICA). Route read-only transactions (@Transactional(readOnly=true)) to the REPLICA DataSource using TransactionSynchronizationManager.isCurrentTransactionReadOnly(). Writes go to PRIMARY. With HikariCP and PgBouncer per replica, you can scale reads horizontally. Monitor replica lag via pg_stat_replication on the primary — alert if lag > acceptable threshold (e.g., 1 second).

Q11. What is write amplification in SSDs and how does it affect database tuning?

SSDs have a write amplification factor: writing 4KB of data may erase and rewrite a 512KB block internally, because SSDs can only write to erased blocks. This means frequent small random writes (index updates, UPDATE-heavy workloads) cause significant internal SSD write amplification, degrading write throughput and SSD lifespan. For database tuning: batch writes where possible, use WAL compression, configure autovacuum to run frequently (reducing large batched cleanups), use BRIN indexes for append-only time-series data (minimal write overhead), and consider WAL archiving vs synchronous_commit tuning.

Q12. What is query plan caching in PostgreSQL and JPA?

PostgreSQL caches prepared statement plans after 5 executions. If the generic plan (parameterized) is cheaper than custom plans, it uses the generic plan. This can cause plan instability: a query with varying parameter selectivity may use a bad cached plan. Force custom plans with SET plan_cache_mode = force_custom_plan. In JPA/Hibernate, queries are compiled to PreparedStatements automatically. Hibernate also has a query plan cache (hibernate.query.plan_cache_max_size). Spring's @Query or CriteriaQuery both use prepared statements. Monitor with pg_prepared_statements.

Q13. What is the difference between B-tree and GIN indexes?

B-tree indexes store keys in sorted order, supporting comparison operators (=, <, >, BETWEEN, LIKE prefix). One index entry per row. GIN (Generalized Inverted Index) stores one index entry per value within a multi-valued column — ideal for JSONB keys, array elements, and full-text search tokens. A JSONB column with 10 keys per document creates 10 GIN index entries per row. GIN is larger and slower to update than B-tree but enables efficient JSONB @> (containment), ? (key exists), and @@ (full-text) operators that B-tree

cannot support.

Q14. How do you tune the PostgreSQL query planner statistics?

The query planner uses column statistics (collected by ANALYZE) to estimate row counts. pg_stats shows these statistics per column. Increase statistics target for columns with high cardinality skewed distributions: ALTER TABLE orders ALTER COLUMN status SET STATISTICS 500; (default 100). Run ANALYZE orders; after to collect. For multi-column statistics (correlated columns): CREATE STATISTICS stat_name (dependencies) ON status, region FROM orders; ANALYZE orders. Multi-variate statistics help the planner understand column correlations.

Q15. What is connection_timeout vs idle_timeout vs max_lifetime in HikariCP?

connection_timeout: maximum time a caller waits for a connection from the pool before throwing SQLException (default 30s). Reduce to 5-10s to fail fast rather than queuing. idle_timeout: time a connection can remain idle in the pool before being closed (default 10 min). min_idle connections are exempt. max_lifetime: maximum lifetime of a connection in the pool (default 30 min). Should be set several minutes less than the database or network firewall connection timeout to avoid stale connection errors. keepalive_time: heartbeat interval to keep firewall-friendly.

Q16. What are the symptoms of a missing database index?

EXPLAIN shows Seq Scan on large tables. High actual time in EXPLAIN ANALYZE for the scan node. pg_stat_statements shows high mean_exec_time or high total_exec_time for queries with WHERE or JOIN predicates. Application-level latency spikes correlating with increasing table size (O(n) growth). CPU spike on database server (table scan is CPU-intensive). Monitor with auto_explain (log_analyze=true, log_min_duration=500ms) to automatically log plans for slow queries.

Q17. How do you handle JSONB performance in PostgreSQL?

JSONB stores JSON in a decomposed binary format for efficient access. Index with GIN for @> (containment) and ? (key exists) queries. For queries on specific JSONB paths, use functional B-tree indexes: CREATE INDEX ON products ((attributes->>'color')); — this allows WHERE attributes->>'color' = 'red' to use the index. Keep JSONB payloads small — very wide JSONB documents increase index size and scan cost. Use generated columns (PostgreSQL 12+) to extract frequently queried JSONB fields into indexed regular columns.

Q18. What does BRIN index provide and when is it the right choice?

BRIN (Block Range Index) stores the min/max value for each contiguous block range (default 128 pages). It is extremely small (typically 1000x smaller than B-tree) and has very low write overhead. It is effective when the indexed column has natural correlation with physical storage order — typically: sequential IDs (BIGSERIAL), timestamps in append-only tables (logs, events, IoT data), created_at columns. It is not effective for randomly distributed data. A Seq Scan for BRIN is faster than a random-access Index Scan for low-selectivity queries.

Q19. What is vacuum and how does it differ from autovacuum?

VACUUM is a PostgreSQL command that reclaims storage occupied by dead tuples, updates the visibility map, and advances the transaction ID counter to prevent wraparound. VACUUM FULL rewrites the entire table (exclusive lock, reclaims space to OS). VACUUM (default) marks dead space for reuse without rewriting. ANALYZE collects column statistics for the query planner. Autovacuum is a background daemon that automatically runs VACUUM and ANALYZE based on thresholds (scale factors). Manual VACUUM is needed after large bulk deletes or when autovacuum thresholds are too conservative.

Q20. How do you implement a database-level performance test?

Use pgbench (built-in PostgreSQL benchmarking tool) for standard TPC-B workloads: pgbench -i -s 100 mydb (initialize 10M rows), pgbench -c 50 -j 4 -T 300 mydb (50 clients, 4 threads, 5 minutes). For custom

query benchmarks, use JMH with a real database and SampleTime mode. Key metrics: transactions per second (TPS), average latency (ms/txn), P99 latency, lock waits (pg_stat_activity with wait_event_type = Lock), autovacuum activity (pg_stat_user_tables). Compare EXPLAIN ANALYZE results before and after index changes.

Q21. How do you use @EntityGraph vs JOIN FETCH in Spring Data JPA?

JOIN FETCH in JPQL: @Query("SELECT o FROM Order o JOIN FETCH o.items JOIN FETCH o.customer WHERE o.id = :id"). Works for ad-hoc complex queries; tightly coupled to JPQL syntax. @EntityGraph: @EntityGraph(attributePaths = {"items", "items.product"}) on the repository method declaration; no custom JPQL needed. Spring Data JPA generates the appropriate fetch join. Both produce a single SQL JOIN query. For multiple bag fetches (two @OneToMany collections), use @EntityGraph with SUBGRAPH or restructure to avoid MultipleBagFetchException by fetching one collection at a time.